

Kod İnceleme Raporu

Yeraltı Savaşları (derin_kazi) — Flutter Uygulaması

Baştan sona kaynak kod taraması: mimari, modül modül işlev analizi, tespit edilen hatalar/riskler ve iyileştirme önerileri.

Proje Adı	derin_kazi ("Yeraltı Savaşları")
Platform	Flutter (Dart)
Toplam Dart Dosyası	56
Toplam Kod Satırı	≈ 16.216 satır
State Yönetimi	flutter_riverpod (StateNotifier)
Analiz Kapsamı	lib/ dizini — tüm .dart dosyaları
Rapor Tarihi	26 Ağustos 2026

1. Yönetici Özeti

"Yeraltı Savaşları", 2D ızgara tabanlı bir kazı/madencilik oyunu etrafında kurulmuş oldukça kapsamlı bir Flutter projesidir. 500 bölüm/10 biyom içeren bir ilerleme sistemi, silah/alet yükseltme (demirci), envanter, günlük görevler, kozmetikler, arkadaş sistemi, "multiplayer" lobi/oda akışı, battle royale modu ve bir AI takım (bot madenci) sistemini bir araya getiriyor. Kod, Riverpod ile MVVM benzeri bir state-management mimarisine ve feature-first klasörlmeye sahip; genel olarak okunabilir, Türkçe yorum/mesajlarla zenginleştirilmiş ve tutarlı bir isimlendirme kullanıyor.

Tarama sonucunda öne çıkan en önemli bulgu şu: projede yer alan "multiplayer" ve rakip oyuncu sistemleri gerçek bir ağ bağlantısı üzerinden çalışmıyor; tamamen cihaz içinde üretilen sahte (mock) verilerle simüle ediliyor (bkz. Bölüm 4.2 ve 5.2). Ayrıca yüklenen arşiv yalnızca **lib/** klasörünü içeriyor; **pubspec.yaml**, **assets/** (ses dosyaları) ve platform klasörleri (android/ios) olmadığı için proje şu haliyle derlenemez durumda (bkz. Bölüm 5.1). Bunların dışında, oyun durumunda (GameState) tutarsız bir **copyWith** deseninden kaynaklanan olası bir UI hata riski tespit edildi (Bölüm 5.3).

Önem	Bulgu Sayısı	Örnek
Kritik	1	Proje derlenebilir/koşulabilir durumda değil (eksik dosyalar)
Yüksek	2	Loot diyalogu state'inin istemsizce sıfırlanma riski; gerçek ağ katmanı yok
Orta	3	22 sessiz (boş) catch bloğu; dev sınıf (2491 satır); sabit kodlanmış metinler
Düşük / Gözlem	4	late alanlar, tekrarlayan switch blokları, i18n altyapısı yok

2. Proje Yapısı ve Mimari

Proje, feature-first bir klasörlleme kullanıyor: her özellik kendi içinde **domain** (modeller/servisler), **application** (Riverpod StateNotifier'ları / iş mantığı) ve **presentation** (ekranlar/widget'lar) alt katmanlarına ayrılmış. Bu, orta-büyük ölçekli bir Flutter projesi için makul ve sürdürülebilir bir yaklaşımdır.

lib/ Klasör Ağacı (özet)

```
lib/
├── main.dart                — Uygulama girişi, yatay mod kilidi, tema
├── core/
│   ├── audio/audio_service.dart — Singleton ses servisi (SFX + BGM)
│   ├── persistence/save_service.dart — SharedPreferences ile kayıt/yükleme
│   └── theme/                — Renk paleti ve MaterialApp teması
├── features/
│   ├── mining/              — Çekirdek oyun (grid, kazı, düşman AI, mağaza, demirci)
│   ├── multiplayer/         — Oda/lobi akışı (yerel simülasyon)
│   ├── battle_royale/       — BR maç fazları, skor tablosu, sonuç ekranı
│   ├── ai_team/             — Bot madenci takımı simülasyonu
│   ├── friends/             — Arkadaş listesi, davet, profil
│   ├── cosmetics/          — Karakter kozmetikleri / skin mağazası
│   ├── quests/              — Günlük/haftalık görev sistemi
│   ├── profile/             — Oyuncu profil ekranı
│   └── settings/            — Ses/ayar ekranı
```

Kullanılan Paketler

Paket	Amaç
flutter_riverpod	State management (StateNotifierProvider / Provider)
shared_preferences	Yerel anahtar-değer kalıcılık (oyuncu, görevler)
audioplayers	SFX ve arka plan müziği oynatma
google_fonts	Özel yazı tipleri

Not: Ağ istekleri (http/dio), gerçek zamanlı iletişim (WebSocket, Firebase, socket.io) veya kimlik doğrulama için herhangi bir pakete rastlanmadı — bu, "multiplayer" özelliğinin gerçek bir sunucu bileşeni olmadığını doğrular.

3. Ölçek Özeti — En Büyük Dosyalar

Kod tabanının büyük kısmı birkaç dosyada yoğunlaşıyor. Aşağıdaki tablo satır sayısına göre en büyük 10 dosyayı listeler:

Dosya	Satır
features/mining/application/game_notifier.dart	2.491
features/profile/presentation/screens/profile_screen.dart	898
features/multiplayer/presentation/screens/main_menu_screen.dart	786
features/mining/presentation/painters/tile_gridPainter.dart	685
features/mining/presentation/screens/forge_screen.dart	663
features/mining/domain/models/player_state_model.dart	661
features/mining/presentation/screens/mining_screen.dart	647
features/friends/presentation/screens/friends_screen.dart	641
features/mining/presentation/screens/weapon_shop_screen.dart	555
features/mining/presentation/widgets/hud_bar.dart	453

game_notifier.dart, projenin toplam kodunun ~%15'ini tek başına oluşturuyor ve grid/kazı, düşman AI, battle royale, envanter, demirci, mağaza, görev takibi ve kayıt gibi çok sayıda sorumluluğu tek bir sınıfta (GameNotifier) barındırıyor (Bölüm 6.2'de detaylandırıldı).

4. Modül Modül Analiz

4.1 core/ — Ortak Altyapı

- AudioService: Singleton pattern ile tek bir ses servisi örneği; SFX çalma, döngülü arka plan müziği, ses/müzik seviyesi kontrolü içeriyor. **isTestMode** bayrağı ile test ortamında ses çağrılarını devre dışı bırakması dikkat çekici, iyi bir pratik.
- SaveService: SharedPreferences üzerinden oyuncu verisi ve görev verisini JSON olarak saklıyor; tüm işlemler try/catch ile sarılmış (hata durumunda sessizce yutuluyor — bkz. Bölüm 5.4).
- AppTheme / AppColors: Karanlık tema, tutarlı bir renk paleti tanımlıyor; küçük ve temiz dosyalar.

4.2 features/mining/ — Çekirdek Oyun Döngüsü

Uygulamanın kalbi burası. GridGenerator, prosedürel olarak bölüm/derinliğe göre bir kutu ızgarası üretiyor (altın, bakır, demir, zümrüt, TNT, gizli mayın, hazine sandığı, boss çekirdeği gibi kutu tipleriyle). EnemySpawner, 500 bölümü 10 biyoma bölüp (Kızıl Toprak Vadisi → Titan'ın Kalbi) her biyom için ayrı düşman/boss havuzları tanımlıyor; bölüm arttıkça HP/hasar çarpanlarıyla ölçekleniyor — iyi kurgulanmış bir içerik ilerleme sistemi. GameNotifier ise oyuncu hareketini, kazı/saldırı hesaplarını (kombo çarpanı, kritik/patlama efektleri), düşman AI'sini (Timer.periodic ile 2.5 sn'de bir), enerji rejenerasyonunu, mağaza/demirci satın alımlarını ve görev ilerlemesini tek bir StateNotifier içinde yönetiyor.

- Kazı hasar sistemi kombo çarpanı (3x → %25, 5x → %50 bonus), aktif alet/silah bonusu ve TNT/gizli mayın gibi alan-etkili özel kutu tipleriyle oldukça detaylı tasarlanmış.
- Demirci (forge) yükseltme sistemi, silah/alet başına 5 seviyeye kadar kademeli maliyet tablosu (altın + maden kaynakları) içeriyor; dengeleme özenli görünüyor.
- **simulateTeammatesAction()** fonksiyonu, oda arkadaşlarının (RemotePlayerModel) hareketini ve kazısını rastgele yönlerde yerel olarak simüle ediyor — gerçek oyuncu girdisi değil.

4.3 features/multiplayer/ — Oda / Lobi Sistemi

LobbyNotifier.createRoom(), rastgele 6 haneli bir oda kodu üretip sabit bir isim listesinden ("KayaKıran", "AltınAvcısı" vb.) sahte takım arkadaşları oluşturuyor.

LobbyNotifier.joinRoom(code) ise girilen kodun sadece 6 karakter olup olmadığını kontrol ediyor; kodun gerçekten var olan bir odaya ait olup olmadığını doğrulamıyor çünkü hiçbir sunucu/istemci iletişimi yok — her geçerli formatlı kod, sabit bir "KaptanMadenci" host'u ile sahte bir odaya bağlanıyor.

4.4 features/battle_royale/ — BR Modu

Geri sayım ekranı, faz durumu modeli (BattlePhaseState — örn. "deathmatch" fazında hasar çarpanı x1.5), skor tablosu paneli ve zafer podyumu diyalogu gibi UI parçalarını içeriyor. Mantık büyük ölçüde GameNotifier içindeki **_checkBattleRoyaleStatus()** ve ilişkili metodlara bağlı.

4.5 features/ai_team/ — Bot Takım Simülasyonu

1-10 oyunculuk bir "takım" kurgusu; oyuncu dışındaki üyeler önceden tanımlı bot profillerinden (AiMinerModel.getPresetMiners()) seçiliyor. Takım büyüklüğüne göre kademeli

bir "takım tamamlama bonusu" ($5 \times (\text{üye}-1)$, 8+ üyede %35 tavan) uygulanıyor. Kod içinde "PDF Bölüm 3.2 & 3.4" gibi bir tasarım dokümanına referanslar var; bu, oyunun bir tasarım belgesinden birebir uygulandığını gösteriyor.

4.6 friends / cosmetics / quests / profile / settings

- friends/: Arkadaş listesi, arama, profil modalı, kozmetik/oda ekranlarına yönlendirme; veriler muhtemelen statik/örnek liste üzerinden geliyor (backend yok).
- cosmetics/: Karakter skin mağazası ekranı; skin verisi **availableSkins** gibi sabit bir listeden geliyor.
- quests/: 12 adet önceden tanımlı haftalık görev (kazı, altın, TNT, boss, prestij vb.), zorluk seviyesine göre ödül ölçekleniyor; haftalık sıfırlama zaman damgası ile takip ediliyor.
- profile/: 898 satırlık büyük bir profil ekranı — istatistikler, rütbe, envanter özetini birleştiriyor.
- settings/: Ses/müzik seviyesi ve genel oyun ayarlarını yöneten standart bir ekran.

5. Tespit Edilen Sorunlar ve Riskler

5.1 [KRİTİK] Proje, verilen haliyle derlenemez

Yüklenen arşiv yalnızca **lib/** klasörünü içeriyor. Aşağıdaki, her Flutter projesi için zorunlu olan öğeler arşivde bulunmuyor:

- **pubspec.yaml** — bağımlılık sürümleri, asset tanımları burada olmalı.
- **assets/sounds/** — AudioService içinde dig.wav, shoot_pistol.wav, explosion.wav, cave_bgm.wav gibi 12'den fazla ses dosyasına referans veriliyor, ancak hiçbir arşivde yok.
- **android/**, **ios/** gibi platform klasörleri.

Bu haliyle proje bir Flutter derleyicisinde çalıştırılmaz; muhtemelen orijinal proje klasöründen yalnızca lib/ alt klasörü sıkıştırılıp paylaşılmış. Tam bir derleme/test için diğer klasörlerin de paylaşılması gerekiyor.

5.2 [YÜKSEK] "Multiplayer" gerçek bir ağ katmanına sahip değil

lib/features/multiplayer/application/lobby_notifier.dart içinde **joinRoom()** fonksiyonu girilen odanın var olup olmadığını hiçbir yerde kontrol etmiyor — sadece kodun 6 karakter olup olmadığına bakıyor ve her zaman sabit bir "KaptanMadenci" adlı sahte host ile bağlanıyor. Benzer şekilde **createRoom()** önceden tanımlı bir isim listesinden ("KayaKıran", "AltınAvcısı"...) sahte takım arkadaşları üretiyor. Projede http/dio/websocket/firebase gibi hiçbir ağ paketine rastlanmadı. Bu, oyunun gerçek bir oyuncu ile eşleşmediğini, sadece "multiplayer görünümlü" tek oyunculu bir deneyim sunduğunu gösteriyor. Bu bilinçli bir tasarım kararıysa sorun değildir, ama kullanıcıya gerçek çok oyunculu bir deneyim vaat ediliyorsa netleştirilmesi/işaretlenmesi önemlidir.

5.3 [YÜKSEK] GameState.copyWith() içinde tutarsız "pendingLoot" alanları

lib/features/mining/application/game_notifier.dart, satır ~216-240: **GameState.copyWith()** içinde çoğu alan **alan ?? this.alan** deseniyle korunurken, **pendingLootName**, **pendingLootMessage** ve **pendingLootWeapon/pendingLootTool** doğrudan parametre değerine atanıyor (aynı ailedeki **pendingLootAmmo** ise doğru şekilde **?? this.x** kullanıyor — bu tutarsızlık kasıtsız bir hatayı işaret ediyor).

Sonuç olarak, bu alanlar yalnızca açıkça geçildiklerinde değerlerini koruyor; **copyWith**'in bu alanlar belirtilmeden çağrıldığı her durumda otomatik olarak null'a dönüyorlar. Oyun, **_startEnemyAiTimer()** ile her 2.5 saniyede bir **_simulateEnemiesAction()** / **simulateTeammatesAction()** çağırıyor ve bu fonksiyonlar da loot alanlarını belirtmeden **state.copyWith(...)** çağırıyor. Pratikte bu, bir kutu açılıp "loot" bulunduğu, UI bu veriyi işleyip diyalogu göstermeden önce arka plandaki periyodik zamanlayıcı tetiklenirse, loot bildiriminin/diyaloğunun sessizce kaybolabileceği anlamına geliyor — zamanlamaya bağlı, aralıklı olarak ortaya çıkan (yeniden üretmesi zor) bir arayüz hatası riski.

Öneri: Bu üç alan da **?? this.alan** deseniyle korunmalı; temizleme işlemi zaten mevcut kodda olduğu gibi (satır ~2167 ve ~2179) açıkça **null** geçilerek yapılmalı.

5.4 [ORTA] Sessiz hata yutma (22 boş catch bloğu)

Kod tabanında **catch (_) {}** deseniyle hataların sessizce yutulduğu 20+ nokta bulundu (SaveService, AudioService, GameNotifier'ın kayıt/yüklemeye metodları dahil). Bu, uygulamanın çökmesini önlemek için savunmacı bir yaklaşım olsa da, geliştirme/hata ayıklama sırasında gerçek sorunların (örn. bozuk kayıt verisi, ses dosyası bulunamaması) fark edilmesini zorlaştırıyor. En azından debug modunda **debugPrint** ile loglanması önerilir.

5.5 [ORTA] GameNotifier tek bir sınıfta çok fazla sorumluluk taşıyor

2.491 satırlık **GameNotifier** sınıfı; ızgara/kazı mantığı, düşman AI, PvP/battle-royale durumu, mağaza satın alımları, demirci yükseltmeleri, görev takibi ve kayıt/yükleme gibi birbirinden bağımsız sorumlulukların tamamını üstleniyor. Bu, tek sorumluluk ilkesini (SRP) zorluyor; test edilebilirliği ve bakımı zorlaştırıyor. Bölümlere ayrılmış ayrı notifier'lar/servisler (örn. ShopService, ForgeService, QuestTracker, CombatEngine) altında birleştirilmiş bir facade önerilir.

5.6 [DÜŞÜK] Sabit kodlanmış Türkçe metinler / i18n altyapısı yok

- Tüm UI metinleri ve oyun içi mesajlar doğrudan Dart kodunda Türkçe string olarak yazılmış (örn. 'Yetersiz Altın!'). Uygulama gelecekte başka dillere çevrilecekse, flutter_localizations / intl paketiyle bir çeviri katmanına geçiş gerekecek.
- Bazı iş kuralları (maliyet tabloları, ödül oranları) kodun içine gömülü sabitler olarak yazılmış; bir JSON/konfig dosyasına taşınması dengeleme (balancing) değişikliklerini kolaylaştırabilir.

6. Güçlü Yönler

İncelemede öne çıkan olumlu noktalar da not edilmelidir:

- Tutarlı mimari: domain / application / presentation ayrımı büyük ölçüde disiplinli şekilde uygulanmış.
- Sağlam kalıcılık katmanı: SaveService ve GameNotifier.loadSavedPlayer() içinde **mounted** kontrolleri ve try/catch koruması doğru yerlerde kullanılmış.
- Zengin ve dengelenmiş içerik sistemi: 500 bölüm / 10 biyom için ayrı düşman havuzları, HP/hasar çarpanları, kademeli demirci maliyet tabloları — belirgin bir tasarım/dengeleme emeği var.
- Widget yaşam döngüsü yönetimi: İncelenen ekranlarda (örn. FriendsScreen) controller'lar **dispose()** içinde düzgünce temizleniyor.
- Singleton ses servisi, test modunda ses çağrılarını no-op yaparak testleri kolaylaştırıyor.
- TODO/FIXME işareti veya **print()** debug çıktısı bulunmadı — kod tabanı "temiz" bırakılmaya özen gösterilmiş.

7. Öncelikli Öneriler

#	Öneri	Öncelik
1	pubspec.yaml, assets/ ve platform klasörlerini (android/ios) ekleyip projeyi github'ten derlenebilir hale getirin.	Küçük
2	GameState.copyWith() içindeki pendingLootName/Message/Weapon/Tool alanları 'Maksimum?? this.x' deseniyle düzeltilsin.	Maksimum
3	Multiplayer'ın yerel simülasyon olduğunu ürün kararı olarak netleştirin; gerçekleştirme isteniyorsa bir backdoor ekleyin.	Küçük
4	Boş catch bloklarına en azından debugPrint ile loglama ekleyin.	Orta
5	GameNotifier'ı Combat/Shop/Forge/Quest gibi alt servislere bölün.	Orta
6	Maliyet/ödül tablolarını kod dışına (JSON/konfig) taşıyarak dengeleme değişiklikleri için kod değişikliği gerektirmeyin.	Düşük

8. Sonuç

"Yeraltı Savaşları", içerik derinliği ve mimari düzen açısından iddialı ve büyük ölçüde iyi kurgulanmış bir Flutter oyun projesi. Bulunan sorunların büyük kısmı (eksik proje dosyaları, bir state alanındaki copyWith tutarsızlığı, sessiz hata yutma) nispeten kolay giderilebilir nitelikte. En kritik ürün kararı, "multiplayer" deneyiminin gerçek bir ağ bağlantısı mı yoksa bilinçli bir yerel simülasyon mu olacağının netleştirilmesidir; bu, projenin sonraki geliştirme yönünü doğrudan etkileyecektir.

Bu rapor, yüklenen kaynak kodun otomatik statik incelemesiyle Claude tarafından oluşturulmuştur.